Argumentation Engine and Strategic Argumentation

Implementation and experimental study of the grounded direct argumentation mechanism

November 2026

1. Overview

This report covers both parts of the assignment:

- **Part 1:** a Python engine for Dung-style abstract argumentation.
- **Part 2:** an implementation and analysis of the grounded direct argumentation mechanism, which asks when agents gain by hiding arguments.

All definitions follow Weiss (ed.), *Multiagent Systems*, 2nd ed., Chapter 5 (Rahwan), pp. 181–201 (PDF pp. 230–250). The code is in `argengine/`, the experiments in `experiments/`, and the demonstration in `demo/` (CLI and Jupyter notebook).

Main findings.

1. Every worked example of the chapter is reproduced exactly (§3), including the two misprints flagged in the brief.
2. The fraction of strategyproof instances falls quickly as the attack probability rises:
   - 0.628 at $p = 0.15$;
   - 0.284 at $p = 0.25$;
   - 0.105 at $p = 0.35$.
3. **No** instance satisfies the Theorem 5.4 condition yet fails strategyproofness. This holds under every reading of "odd-length path", and in the 3000 random instances, an exhaustive check of all 3-agent instances with $n \leq 4$ and a 300 000-instance stress test. §5.2 proves why: a sharpened form of Theorem 5.4 holds.
4. Strategyproof instances that violate the condition are common: 876 of 3000, and 82–97% of all strategyproof instances. Almost all of them (856) are explained by an *offset* effect: hiding an argument does free one of the agent's own arguments, but it costs at least as much, usually the hidden argument itself.

2. Design of the engine

2.1 Representation

`ArgumentationFramework(arguments, attacks)` stores $\langle \mathcal{A}, \rightharpoonup \rangle$ (Def. 5.1).

- The public API takes and returns ordinary Python sets of argument names.
- Internally, argument i is bit i, and a set $S \subseteq \mathcal{A}$ is an integer bitmask.
- For every argument the class precomputes two masks: its attackers α^- and its targets.

With these masks, the basic operations of Def. 5.2 are a few bit operations:

- S^+ is the OR of the target masks of S;
- *conflict-free* is `plus(S) & S == 0`;
- S *defends* α is `attackers[α] & ~plus(S) == 0`.

Because every subset of $\mathcal{A}$ is an integer in $[0, 2^n)$, subsets can be enumerated and cached cheaply. Part 2 relies on this.

2.2 Assignment items and where they live

Item	Function	Definition used
1 Framework class	`ArgumentationFramework`	Def. 5.1
2 Conflict-free	`is_conflict_free(af, S)`	$S \cap S^+ = \emptyset$
3 Defence	`defends(af, S, a)`	$\alpha^- \subseteq S^+$
4 Characteristic function	`characteristic_function(af, S)`	$F(S) = \{\alpha \mid S \text{ defends } \alpha\}$
5 Admissible / complete	`is_admissible`, `is_complete_extension`	$S \subseteq F(S)$ / $S = F(S)$, both conflict-free
6A Grounded	`grounded_extension`, `grounded_trace`	least fixed point, iterate F from $\emptyset$
6B Complete	`complete_extensions`	all fixed points of F that are conflict-free
6C Preferred / stable / semi-stable	`preferred_extensions`, `stable_extensions`, `semi_stable_extensions`, `all_extensions`	Def. 5.7
6D Labellings	`all_labellings`, `is_legal`	Def. 5.6, Table 5.1
6E Acceptance status	`acceptance_status(af, a, semantics)`	Def. 5.8

2.3 Algorithms

- **Grounded.** Start from $\emptyset$ and apply F until the set stops changing. F is monotone, so this reaches the least fixed point in at most n steps.
- **Complete.** Every complete extension E contains the grounded extension G, and since E is conflict-free it contains nothing in G^+. So the search only tries $G \cup X$ for subsets X of the remaining free arguments $\mathcal{A} \setminus (G \cup G^+)$, and keeps those that are conflict-free with $F(S) = S$.
- **Preferred:** the $\subseteq$-maximal complete extensions. A test checks that these equal the maximal admissible sets.
- **Stable:** complete extensions with $S^+ = \mathcal{A} \setminus S$.
- **Semi-stable:** complete extensions whose range $S \cup S^+$ is $\subseteq$-maximal.
- **Labellings.** Each labelling is obtained from a complete extension with in $= S$, out $= S^+$, undec $=$ the rest (Caminada's bijection). A brute-force enumeration of all 3^n labellings confirms the bijection in tests. Table 5.1 is implemented as selectors on the labellings (minimal in, maximal in, minimal undec, empty undec), and a test checks that they return exactly the corresponding extensions.
- **Acceptance status.**
  - *Skeptically accepted:* the argument is in every extension.
  - *Credulously accepted:* it is in at least one extension.
  - *Rejected:* it is in none.

The function reports the strongest status that applies. One edge case needed a decision: when a semantics has **no** extension (stable semantics on an odd cycle), "in every extension" is vacuously true. We report the argument as *rejected* and flag `n_extensions = 0`, because with no extension there is nothing to accept.

Enumeration is exponential, but it is instant for the sizes used here: the whole test suite runs in about 2 s, and the 3000-instance experiment in 2.5 s.

3. Validation on the book's examples

Each row below is an automated test in `tests/test_book_examples.py` or `tests/test_strategic.py`. Greek α_k is written `ak` in the code.

Example	Book statement	Computed
Fig 5.1, Ex 5.5	$\emptyset, \{\alpha_3\}, \{\alpha_5\}, \{\alpha_3, \alpha_5\}, \{\alpha_1, \alpha_3, \alpha_5\}$ are admissible	✓
Fig 5.1, Ex 5.6	$F(\{\alpha_3\}) = F(\{\alpha_5\}) = \{\alpha_3, \alpha_5\}$; the only complete extension is $\{\alpha_1, \alpha_3, \alpha_5\}$	✓
Fig 5.1, p.184	$F^1(\emptyset) = \{\alpha_3, \alpha_5\}$, $F^2(\emptyset) = F^3(\emptyset) = \{\alpha_1, \alpha_3, \alpha_5\}$	✓
Fig 5.2, Ex 5.7	complete: $\{\alpha_3\}, \{\alpha_1, \alpha_3\}, \{\alpha_2, \alpha_3\}$; three labellings L^G, L_1, L_2	✓
Fig 5.2, Ex 5.8	grounded $\{\alpha_3\}$; preferred $\{\alpha_1, \alpha_3\}, \{\alpha_2, \alpha_3\}$	✓
Fig 5.3, p.188	grounded $\{a, c, e\}$	✓
Ex 5.10	truthful outcome $\{\alpha_1, \alpha_3\}$; x is better off revealing $\{\alpha_4, \alpha_5\}$, giving outcome $\{\alpha_2, \alpha_4, \alpha_5\}$ and utility 2; not strategyproof	✓
Ex 5.11	truthful outcome $\{\alpha_1, \alpha_4, \alpha_5, \alpha_6\}$; truth-telling dominant for x although $\alpha_1 \hookrightarrow \alpha_4, \alpha_5$	✓

The book has no example in which preferred, semi-stable and stable differ, so three small frameworks were added:

- an odd 3-cycle, which has no stable extension;
- $a \rightleftharpoons b$, $b \rightharpoonup c$ with an odd cycle $c \rightharpoonup d \rightharpoonup e \rightharpoonup c$, where preferred $= \{a\}, \{b, d\}$ but stable $=$ semi-stable $= \{b, d\}$;
- a framework with semi-stable extensions but no stable one.

Property-based tests (`hypothesis`, hundreds of random frameworks with $n \leq 6$) check the following:

- grounded $\subseteq$ every complete extension;
- stable $\subseteq$ semi-stable $\subseteq$ preferred $\subseteq$ complete;
- semi-stable $=$ stable whenever a stable extension exists;
- preferred $=$ maximal admissible;
- F is monotone;
- the labelling bijection holds and agrees with the 3^n brute force.

Misprints confirmed.

- *Ex. 5.10.* The utilities of y and z are printed with each other's argument sets. The correct values are $u_y(o, \{\alpha_2\}) = 0$ and $u_z(o, \{\alpha_3\}) = 1$.
- *After Ex. 5.11.* The argument that cancels x's indirect self-defeat is α_6, not α_5. The engine shows this directly. On the same graph, if x does not own α_6, the mechanism is not strategyproof again. When x does own α_6, revealing it defeats α_3 and protects α_4, α_5 whatever x does with α_1.

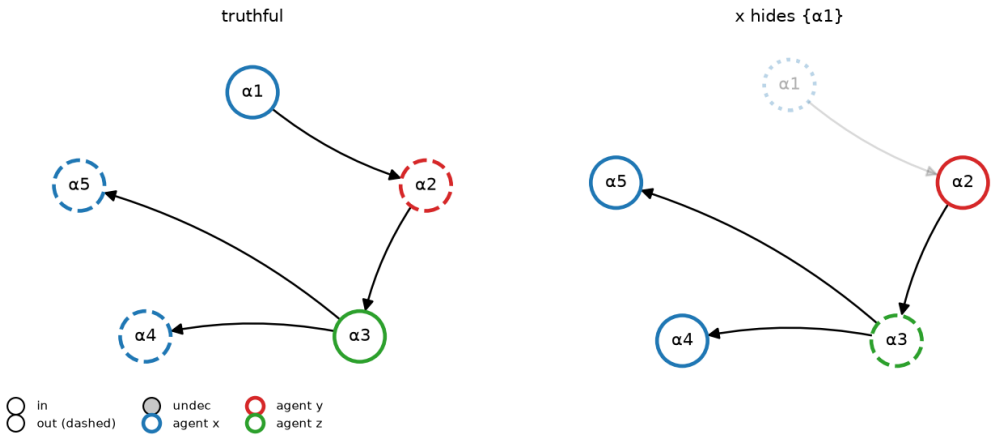


Figure 1: Example 5.10 (Fig. 5.4) as drawn by the engine. Left: full revelation. Right: x withholds α_1. Solid border = in, dashed = out, grey = undec; border colour = owner.

4. Part 2: the grounded direct mechanism

4.1 Model

- The global attack relation R over $\mathcal{A}$ is common knowledge.
- Agent i's type is $\mathcal{A}_i \subseteq \mathcal{A}$.
- Agents can only lie by hiding arguments, so a report is any $\mathcal{A}_i^\circ \subseteq \mathcal{A}_i$.
- The outcome is $g(\mathcal{A}_1^\circ, \dots, \mathcal{A}_I^\circ) = GE(\langle \bigcup_i \mathcal{A}_i^\circ, R \rangle)$ (Def. 5.21).
- Utility is $u_i(o) = |\mathcal{A}_i \cap o|$ (Def. 5.22).

4.2 Functions

Item	Function	What it does
1	`grounded_mechanism(R, reports)`	grounded extension of the graph induced by $\bigcup$ reports
2	`best_responses(R, types, i, others_reports)`	enumerates all $2^{\lvert\mathcal{A}_i\rvert}$ reports of agent i and returns **every** utility-maximising report, plus whether truth-telling is among them
3	`is_strategyproof(R, types)`	the Theorem 5.3 condition: for every agent i, every report profile of the others, and every $S \subseteq \mathcal{A}_i$, $\lvert\mathcal{A}_i \cap GE(\mathcal{A}_i \cup \mathcal{A}_{-i})\rvert \geq \lvert\mathcal{A}_i \cap GE((\mathcal{A}_i \setminus S) \cup \mathcal{A}_{-i})\rvert$; returns witnesses (profitable deviations) when it fails
4	`theorem_5_4_condition(R, types, mode, include_self)`	checks that no agent holds α, β with an odd-length directed path $\alpha \rightsquigarrow \beta$ in the full graph; returns the violating paths

Efficiency. The outcome depends only on the *union* of the reports, which is a subset of $\mathcal{A}$. `MechanismInstance` therefore caches the grounded extension of each revealed set, at most $2^n = 256$ of them for $n = 8$. The union of the other agents' reports ranges over every subset of the union of their types, so the strategyproofness check needs at most $3 \cdot 2^n$ cache lookups per instance. A literal brute-force implementation of Theorem 5.3 that enumerates every report profile agent by agent is used as a cross-check in the tests.

4.3 Cycles and the meaning of "odd-length path"

On a cyclic graph, "an odd-length path from α to β" can be read in two ways, and both are implemented.

- **Walk** (vertices may repeat). This uses a BFS over the parity-doubled graph, whose states are (vertex, parity of the length so far). Each state is visited once, so cycles cannot cause non-termination. An odd cycle simply lets a vertex be reached with both parities. Cost: $O(n(n + |R|))$.
- **Simple path** (no repeated vertex). This uses a DFS over simple paths. When $\alpha = \beta$, it looks for an odd simple cycle through α. In general this is exponential (finding an odd simple path in a digraph is NP-hard), but it is instant for $n \leq 8$.

Two further choices are covered:

- $\alpha = \beta$**.** The theorem quantifies over all $\alpha_1, \alpha_2 \in \mathcal{A}_i$, so we also run the check including $\alpha = \beta$: an odd cycle through one of the agent's own arguments, or a self-attack.
- **Direct attacks.** A direct attack inside a type is the length-1 case, which covers the "conflict-free type" clause of the theorem.

Every simple-path violation is also a walk violation, and in acyclic graphs the two readings coincide. Both facts are property-tested.

Primary reading. "Path" in the standard graph-theory sense means a *simple path*, and $\alpha = \beta$ is allowed by the quantifier. This literal reading is used as the primary one, and the tables in the summary report all four combinations.

5. Experiments

Setup.

- 3 agents and $n \sim U\{6, 7, 8\}$ arguments.
- Each ordered pair (a, b) with $a \neq b$ is an attack independently with probability $p \in \{0.15, 0.25, 0.35\}$.
- Each argument is given to one of the 3 agents uniformly at random, so types are disjoint and may be empty.
- 1000 instances per p, 3000 in total, with seed 2026 and a separate random stream for each p.

The full run takes 2.5 s. Raw data is in `experiments/results/instances.jsonl` and `instances.csv`, and the tables are in `summary.md`.

5.1 Q1: fraction of strategyproof instances

p	strategyproof	fraction	95% Wilson CI	Thm 5.4 condition holds	acyclic graphs	mean $\lvert R\rvert$
0.15	628 / 1000	**0.628**	[0.598, 0.657]	0.111	0.527	6.5
0.25	284 / 1000	**0.284**	[0.257, 0.313]	0.027	0.158	10.5
0.35	105 / 1000	**0.105**	[0.087, 0.126]	0.003	0.035	15.0

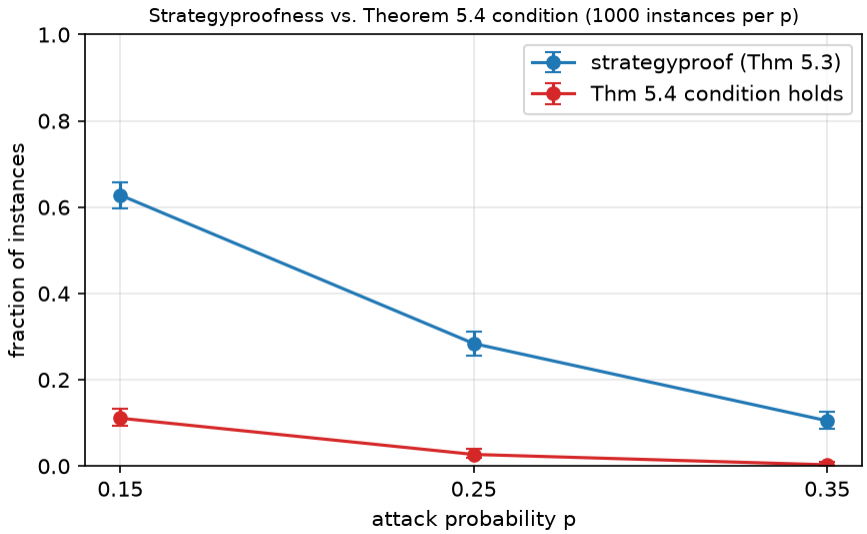


Figure 2: Fraction of strategyproof instances, and of instances satisfying the Theorem 5.4 condition, against p (95% Wilson intervals).

Strategyproofness becomes rarer as p grows, and also as n grows. At $p = 0.25$ the fraction is 0.43, 0.28 and 0.14 for $n = 6, 7, 8$; see `fig_sp_by_n.png`. Denser graphs contain more chains through which an agent's own arguments attack each other indirectly.

Cycles matter a great deal:

- acyclic instances are strategyproof about 87–89% of the time at every p;
- instances with a cycle are strategyproof only 34%, 18% and 8% of the time at the three values of p.

The Theorem 5.4 condition is far more restrictive than strategyproofness itself. At $p = 0.35$ only 3 instances satisfy it, while 105 are strategyproof.

5.2 Q2: condition holds but not strategyproof (none)

reading of the condition	$p = 0.15$	$p = 0.25$	$p = 0.35$
odd simple path, $\alpha = \beta$ allowed (primary)	0	0	0
odd walk, $\alpha = \beta$ allowed	0	0	0
odd simple path, $\alpha \neq \beta$	0	0	0
odd walk, $\alpha \neq \beta$	0	0	0

The weakest reading (simple paths, $\alpha \neq \beta$) was also checked beyond the 3000 instances, again with no counterexample:

- **Exhaustively** over every 3-agent instance with $n \leq 4$, self-attacks allowed: 463 866 instances satisfy the condition.
- On **300 000** further random instances with $n \in [3, 9]$, $p \in [0.1, 0.5]$ and self-attacks allowed: 55 740 satisfy the condition.

This is not luck. The following sharpened version of Theorem 5.4 holds; it is also tested in `tests/test_strategic.py`.

Lemma. Let $M \subseteq \mathcal{A}$, $S \subseteq M$, $G = GE(M)$ and $G' = GE(M \setminus S)$. If $\beta \in G' \setminus G$, then some $\alpha \in S$ has an odd-length *simple* path $\alpha \rightsquigarrow \beta$ in the graph induced by M.

Proof. Let $G'_k = F^k_{M \setminus S}(\emptyset)$, and induct on the first stage k at which $\beta \in G'_k$.

1. Since $\beta \in M$ but $\beta \notin G = F_M(G)$, some attacker $\gamma \in M$ of β is not attacked by G.
2. If $\gamma \in S$, then $\gamma \rightharpoonup \beta$ is an odd path of length 1, and we are done.
3. Otherwise $\gamma \in M \setminus S$. Because $\beta \in G'_k$, γ is attacked by some $\delta \in G'_{k-1}$. Moreover $\delta \notin G$, since otherwise γ would be attacked by G. By induction there is an odd chain from some $\alpha \in S$ to δ. Appending $\delta \rightharpoonup \gamma \rightharpoonup \beta$ adds two edges, so the chain stays odd.
4. The chain is α followed by alternating vertices $\delta_j \in G'$ and $\gamma_j \notin G'$, ending at β.
   - $\gamma_j \notin G'$ because it is attacked by a member of the conflict-free set G'.
   - The δ_j have strictly decreasing stages, so they are distinct from each other and from β.
   - $\alpha \in S$ cannot occur again in $M \setminus S$.

   So only a γ can repeat. Cutting the segment between two occurrences of the same γ removes an even number of edges, so the parity stays odd and the chain becomes a simple path. □

Theorem (sharpened 5.4). Suppose no agent holds two *distinct* arguments $\alpha \neq \beta$ with an odd simple path $\alpha \rightsquigarrow \beta$ in the full graph. Then the grounded direct mechanism is strategyproof for acceptability-maximising agents.

Proof. Suppose agent i gains by hiding $S \subseteq \mathcal{A}_i$ against $\mathcal{A}_{-i}$, and let $M = \mathcal{A}_i \cup \mathcal{A}_{-i}$.

1. Then $|\mathcal{A}_i \cap G'| > |\mathcal{A}_i \cap G|$, so some $\beta \in \mathcal{A}_i \cap (G' \setminus G)$.
2. By the lemma there is $\alpha \in S \subseteq \mathcal{A}_i$ with an odd simple path to β, and $\alpha \neq \beta$ because $\beta \in M \setminus S$.
3. The induced graph is a subgraph of the full graph, so the path is also in the full graph. This contradicts the assumption. □

Each of the four readings in the table is at least as strong as this one: a walk condition forbids more pairs, and allowing $\alpha = \beta$ forbids more. So all four are sufficient, and **Q2 is necessarily empty**. Cycles do not break the theorem.

On the data, the lemma's prediction was confirmed for all 1983 manipulable instances: every argument a manipulator gains is reachable from one of its hidden arguments by an odd simple path. The proof also shows that the $\alpha = \beta$ part of the condition is unnecessary. In our data it changes the verdict in only 4 instances, all strategyproof and all of the "dormant" kind described below.

5.3 Q3: strategyproof although the condition fails

p	SP but condition fails	share of all SP instances	offset	self-neutralised	dormant	only direct attacks inside a type
0.15	517	0.823	506	2	9	354
0.25	257	0.905	249	0	8	100
0.35	102	0.971	101	0	1	18

These instances are the norm rather than the exception: Theorem 5.4 is sufficient but far from necessary, while Theorem 5.3 is the exact characterisation. The condition only asks whether an odd path *could* exist in the full graph. It ignores two things:

- whether hiding would actually make the target **in**, given the rest of the graph;
- whether the gain outweighs what the agent loses.

`MechanismInstance.classify_agent` sorts every violating agent into three classes, by checking every revealed set $M \subseteq \mathcal{A}$ and every hidden $S \subseteq \mathcal{A}_i \cap M$.

(a) Offset: 856 instances, 1195 agents. Here hiding does raise one of the agent's arguments to **in**, even compared with telling the truth, but the agent always loses at least as many accepted arguments.

- In 1048 of the 1195 agents the best such deviation is an *exact swap* with net change 0. Typically the hidden argument was itself **in** and is lost. Truth-telling is then weakly dominant but not strictly.
- In 472 instances, every violation is a direct attack inside a type, so the type is not even conflict-free. For example, an agent owning $\alpha_1 \rightharpoonup \alpha_4$ can trade α_1 for α_4 but never gain.
- An indirect example is instance #254: $x = \{\alpha_2, \alpha_4\}$ with the path $\alpha_2 \rightharpoonup \alpha_6 \rightharpoonup \alpha_1 \rightharpoonup \alpha_4$. When the others reveal α_1, α_6, hiding α_2 lets α_6 in, which defeats α_1 and frees α_4. But α_2 itself, which was **in**, is lost.

Offset is the dominant class at every p (97–99% of Q3 instances). Example 5.10 is exactly the case where the swap is *not* balanced: hiding the single argument α_1 frees two arguments, α_4 and α_5.

(b) Self-neutralised: 2 instances, 9 agents. This is the generalised Example 5.11 pattern. Hiding can lift one of the agent's own arguments, but only if the agent also withholds another of its arguments. When it reveals the rest of its type, that other argument blocks the gain, as α_6 does in Ex. 5.11.

- In instance #296, $z = \{\alpha_1, \alpha_2, \alpha_4\}$ has the odd path $\alpha_4 \rightharpoonup \alpha_5 \rightharpoonup \alpha_6 \rightharpoonup \alpha_1$. Hiding α_4 would free α_1, but only while z withholds α_2. Once α_2 is revealed, $\alpha_2 \rightharpoonup \alpha_6$ keeps α_6 out regardless (Fig. below).
- In #900, y's α_7 plays the same protective role for α_4.

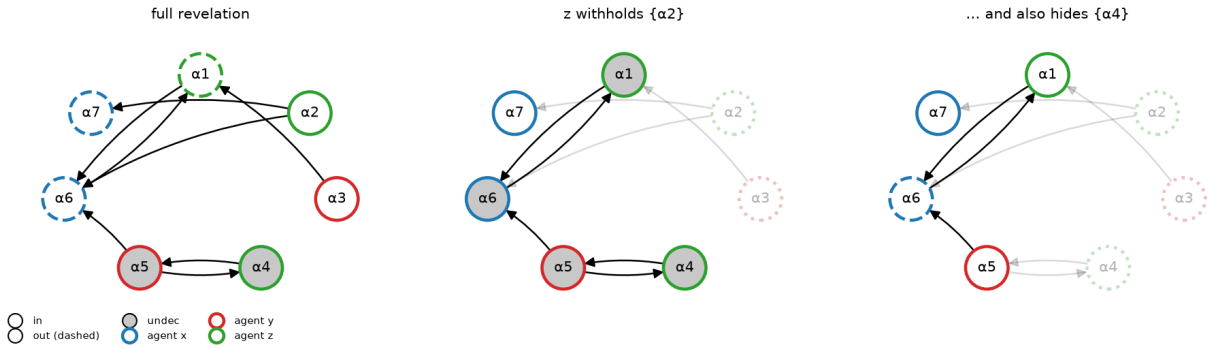


Figure 3: Instance #296 (self-neutralised). Left: full revelation. Middle: z withholds α_2 (and y hides α_3). Right: only then does hiding α_4 turn α_1 in. With α_2 revealed, α_6 is always out and α_4 is irrelevant to α_1.

(c) Dormant: 18 instances, 225 agents. Here the odd path never matters: no revealed set and no hidden set ever raises any of the agent's arguments to **in**. We observed three mechanisms.

- *Odd cycle through a single owned argument ($\alpha = \beta$).* In instance #203 the 3-cycle $\alpha_5 \rightharpoonup \alpha_3 \rightharpoonup \alpha_1 \rightharpoonup \alpha_5$ is split one argument per agent. Hiding your own cycle argument removes that very argument, so it can never help. This is why the lemma needs $\alpha \neq \beta$.
- *The odd cycle swallows the path.* In #970, z's path $\alpha_2 \rightharpoonup \alpha_5 \rightharpoonup \alpha_4 \rightharpoonup \alpha_3$ runs into the odd cycle $\alpha_3 \rightharpoonup \alpha_5 \rightharpoonup \alpha_4 \rightharpoonup \alpha_3$, which z's own α_3 closes.
  - Whenever α_4 and α_5 are both present, the cycle keeps α_3 undec whatever α_2 does.
  - When either is missing, α_3's status does not depend on α_2.
- *Freed to undec, not to in.* In #900, x's path $\alpha_3 \rightharpoonup \alpha_1 \rightharpoonup \alpha_4 \rightharpoonup \alpha_6$ passes through the mutual attack $\alpha_1 \rightleftharpoons \alpha_4$. Hiding α_3 can only turn α_6 from out into **undec**, because α_1 and α_4 then block each other.

Utility counts only accepted (in) arguments, so the change is worth nothing.

The `experiments/results/examples/` folder has drawings and step-by-step explanations for each class (`examples.md`).

5.4 Discussion

- **Why strategyproofness falls with p.** Manipulation needs an odd path from a hidden argument to a gained one (the lemma). As p grows, more pairs of the agent's own arguments are joined by odd paths, and more of those paths are "live" rather than buried in cycles.
- **Cycles cut both ways.**
  - Cyclic graphs are much less often strategyproof, because they create more odd paths.
  - But odd cycles also make many arguments undec, which neutralises potential gains: this is the dormant class.
  - The theorem itself is unaffected by cycles (§5.2).
- **Walk vs simple path.** On our data the two readings agree on all 3000 instances. A walk violation that is not a simple-path violation needs an odd cycle on the way. In practice such graphs almost always contain a simple odd path between the same agent's arguments anyway. The difference does exist (see the unit test `test_walk_mode_shows_difference_from_simple`), but by the theorem above it can never matter for sufficiency.
- **Limitations.**
  - Enumeration is exponential in n. That is fine for $n \leq\sim 20$ in Part 1 and $n \leq\sim 14$ in Part 2.
  - Types are disjoint in the experiments; the code supports overlapping types.
  - Only lying by hiding arguments is modelled, as in the book.

6. Reproducing

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest                          # 161 tests
.venv/bin/python -m argengine.demo                  # CLI demonstration
.venv/bin/python experiments/run_experiments.py     # 3 x 1000 instances, seed 2026
.venv/bin/python experiments/analyse.py             # tables, plots, examples
```